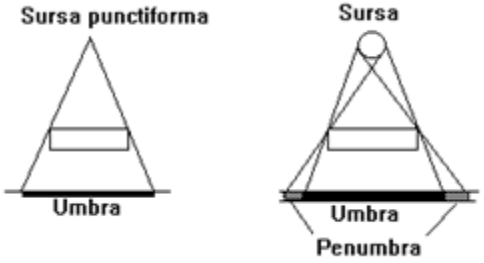
Redarea umbrelor în imagini

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

INTRODUCEREA UMBRELOR IN IMAGINI(1)

- Atunci când un observator priveşte o scenă 3D luminată de o sursă de lumină dintr-o poziţie diferită de aceea a sursei de lumină, va vedea umbrele produse de obiectele scenei.
- Umbrele au o contribuţie însemnată la realismul imaginii, îmbunătăţind percepţia profunzimii.



In general, se considera numai umbra pura: calcule mai simple.

Volumul de calcule depinde şi de poziţia sursei de lumină:

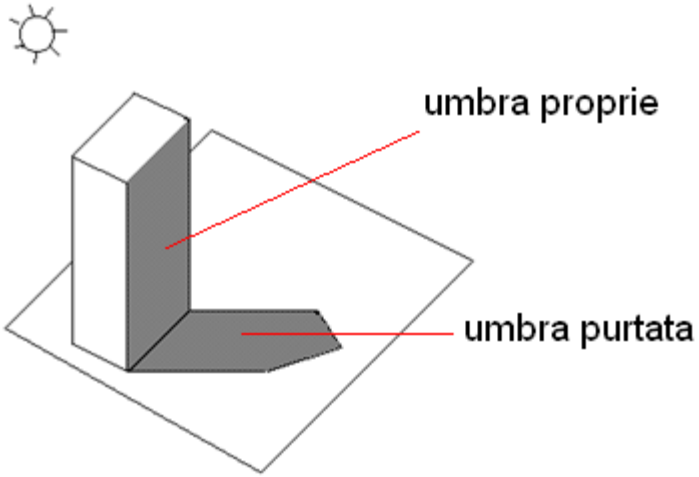
- Dacă sursa este la infinit calculele sunt mai simple.
- In cazul unei surse situate la distanţă finită dar în afara volumului vizual, este necesară o proiecţie perspectivă din poziţia sursei.
- Cazul cel mai dificil este acela în care sursa este situată în volumul vizual.

INTRODUCEREA UMBRELOR IN IMAGINI(2)

- Problema determinării umbrelor este similară aceleia de determinare a părților nevizibile ale unei scene 3D: partile scenei 3D care nu sunt vizibile din poziţia sursei de lumină sunt în umbră.

  → Crearea unei imagini cu umbre presupune rezolvarea de două ori a problemei suprafeţelor nevizibile: o dată privind scena din poziţia sursei de lumină, a doua oară privind-o din poziţia observatorului.

- Sunt două tipuri de umbre: **umbre proprii** şi **umbre purtate**.



POLIGOANE DETALIU DE UMBRA (1)

–Umbrele proprii sunt generate de obiectul însuşi, care împiedică lumina să ajungă la unele dintre feţele sale: feţele umbrite de umbra proprie sunt feţele auto-obturate atunci când scena este văzută din poziţia sursei de lumină.

–Umbra purtată este umbra pe care un obiect o produce pe alte părţi ale scenei, la care lumina nu ajunge din cauza obiectului.

Poligoane detaliu

❑ Umbrele purtate se pot determina proiectând în scenă, din poziţia sursei, toate poligoanele (feţele) neumbrite de umbra proprie.

- Rezultă un set de **poligoane de umbră**, care se adaugă la reprezentarea scenei.

❑ Atât fețele umbrite de umbra proprie cat şi poligoanele de umbra se folosesc ca poligoane-detaliu la redarea scenei 3D.

❑ Poligoanele detaliu sunt poligoane coplanare cu poligoanele scenei, care se folosesc în calculul culorii fragmentelor.

POLIGOANE DETALIU DE UMBRA (2)

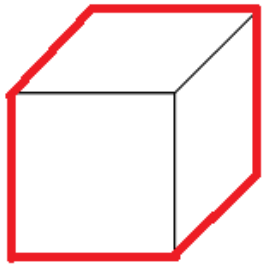
- Poligoanele de umbra au o culoare specifica umbrei pure.
- Poligoanele de umbra se trimit in banda grafica dupa poligoanele scenei, care sunt vizibile din poziţia observatorului, cu testul de adancime dezactivat (fragmentele poligoanelor de umbra suprascriu unele dintre fragmentele vizibile) .

- Pot fi generate mai multe vederi ale scenei fără a recalcula umbrele, deoarece umbrele depind numai de poziţia sursei (surselor) de lumină.

- Numărul de poligoane de umbră este mai mic,

dacă în loc să se proiecteze fiecare faţă luminată de sursă,

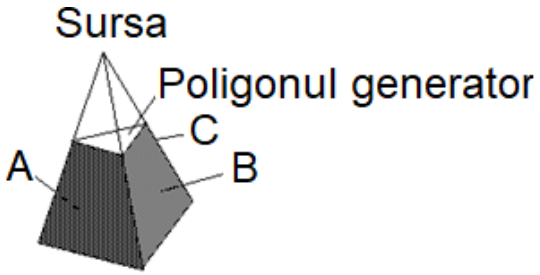
se proiectează silueta fiecărui obiect văzută din poziţia sursei de lumină.



- Calculul poligoanelor de umbra este destul de complicat, de aceea in prezent se folosesc alte metode pentru introducerea umbrelor in imagini.

VOLUME DE UMBRA (1)

- Sursa de lumină este considerată punctiformă iar scena 3D alcatuita din poligoane.
- Un **volum de umbră** este definit de o sursă de lumină şi un poligon luminat (vizibil din poziţia sursei de lumină), pe care-l vom numi **poligonul generator**.



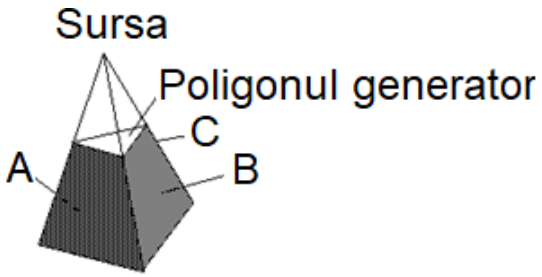
- Volumul de umbra determinat de o sursă şi un poligon generator este delimitat de o faţă care reprezintă poligonul generator scalat. Această faţă este situată la o distanţă faţă de sursă dincolo de care intensitatea luminii sursei este neglijabilă, deci orice punct aflat dincolo de această limită este umbrit.
- Volumul de umbră poate fi decupat la marginile volumului vizual.

VOLUME DE UMBRA (2)

❑ Feţele laterale ale volumului sunt numite **poligoane de umbră**. Un poligon de umbră este determinat de o latură a poligonului generator şi cele două drepte care pleacă din sursa de lumină, fiecare trecând printr-un vârf al laturii.

❑ Normalele la feţele laterale punctează înspre exteriorul volumului.

❑ Poligoanele de umbră se folosesc pentru determinarea umbririi produse de poligonul generator în scenă.



Notăm cu :

– PUV poligoanele de umbră care sunt vizibile din poziţia observatorului (A şi B în figura) şi cu

– PUN poligoanele de umbră care nu sunt vizibile din poziţia observatorului (poligonul C).

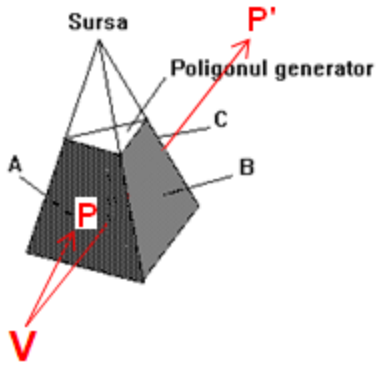
➢ Clasificarea poligoanelor de umbra in PUV si PUN se poate face pe baza normalelor la poligoane si a pozitiei observatorului (back face culling).

VOLUME DE UMBRA (3)

Fie

– un punct P al unei suprafeţe din scena 3D şi

– VP vectorul din poziţia observatorului (V) în punctul P.



Punctul P este umbrit dacă numărul de poligoane de tip PUV intersectate de vectorul VP este mai mare decât numărul de poligoane de tip PUN intersectate de vector.

Acesta este singurul caz, atunci când punctul V nu este în umbră.

❑ In general, pentru a determina dacă un punct este în umbră, se poate folosi un contor în care iniţial se memorează numărul de volume de umbră care conţin poziţia observatorului.

– Se asociază poligoanelor de tip PUV valoarea +1 iar celor de tip PUN valoarea -1.

– Atunci când vectorul VP traversează un poligon de umbră, se adună la contor valoarea asociată poligonului.

– Punctul P este umbrit dacă valoarea contorului este pozitivă în P.

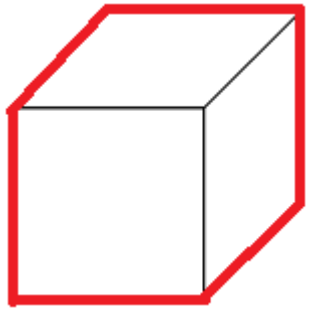
VOLUME DE UMBRA (4)

- Volumul de calcule presupus de acest algoritm poate fi redus dacă în loc să se calculeze volumul de umbră pentru fiecare poligon vizibil din poziţia sursei, se calculează un singur volum de umbră pentru o suprafaţă poliedrală.

In acest scop, se determină poligoanele de umbră numai pentru laturile care fac parte din silueta suprafeţei, văzută din poziţia sursei de lumina.

- ❑ Silueta unei suprafeţe, corespunzătoare unui punct de observare, este un set conectat de laturi care aparţin poligoanelor vizibile din punctul de observare.
- ❑ O latură de siluetă este fie o latură de margine a unei suprafeţe deschise, fie o latură care separă un poligon vizibil de unul nevizibil.

- Pentru determinarea laturilor de siluetă, este necesar să se folosească o structură de date care reflectă adiacenţa poligoanelor. Cunoscându-se poligonul adiacent pe fiecare latură a fiecărui poligon vizibil din poziţia sursei, se pot determina rapid laturile de siluetă.



VOLUME DE UMBRA (5)

Implementarea metodei folosind stencil-buffer(1)

- Buffer-ul de marcaje (stencil-buffer) este prezent pe placile grafice actuale (in memoria GPU), alaturi de buffer-ul imagine (frame-buffer) si buffer-ul de adancime (z-buffer).

– este o memorie de numere intregi asociata pixelilor imaginii, de regula un byte/pixel;

– poate fi utilizat pentru a implementa diferite operatii grafice, de regula in combinatie cu z-buffer, cum este si cazul implementarii volumelor de umbra:

– valorile din stencil-buffer pot fi incrementate/ decrementate automat pentru fiecare fragment care trece/ nu trece testul de adancime.

- Exista mai multe metode de a utiliza stencil-buffer pentru implementarea metodei volumelor de umbre.

VOLUME DE UMBRA (6)

Implementarea metodei folosind stencil-buffer(2)

- In metoda prezentata in continuare, se creaza in stencil-buffer o masca a umbrelor din imagine: orice pixel de umbra, (x,y), va fi reprezentat in stencil-buffer printr-o valoare $\neq 0$.

- **Pixelii de umbra sunt afisati in culorile fragmentelor vizibile, calculate cu formula.**

Cfragment = Cemisiva + Cambianta = ke + ka*globalAmbient

ke, ka – proprietati de material

– Se presupune ca s-au calculat poligoanele de umbra.

– Conturul fiecarui poligon de umbra trebuie sa fie orientat in sens trigonometric atunci cand este privit din exteriorul volumului de umbra. In acest fel, poligoanele PUV sunt cele “din faţa” observatorului, iar poligoanele PUN sunt cele “din spate”.

VOLUME DE UMBRA (7)

Implementarea metodei folosind stencil-buffer(3)

Consideram cazul in care observatorul nu este in umbra.

Pasul 1

Initializeaza z-buffer si stencil-buffer; // nu trebuie programate, acestea sunt
// initializate in mod automat pentru fiecare cadru imagine

Activeaza scrierea in z-buffer si dezactiveaza scrierea in stencil-buffer ; //implicite

*** Rasterizeaza scena (folosindu-se algoritmul z-buffer) fara lumini**

Efectul:

– in frame-buffer se obtine imaginea scenei in umbra (fara lumini). Culoarea fiecarui pixel este:

Cpixel = culoarea emisiva + culoarea ambientala;

La sfarsitul acestui pas, z-buffer contine coordonatele z ale fragmentelor vizibile in imagine.

VOLUME DE UMBRA (8)

Implementarea metodei folosind stencil-buffer(4)

Pasul 2

glDepthMask(GL_FALSE); // Dezactiveaza scrierea in z-buffer

glColorMask(GL_FALSE, GL_FALSE, GL_FALSE, GL_FALSE); // Dezactiveaza scrierea in frame-buffer

glFrontFace(GL_CCW);// feţele “din faţă” sunt orientate trigonometric

glCullFace(GL_BACK);//se vor elimina feţele din spate (nu sunt rasterizate poligoanele PUN)

glStencilFunc(GL_ALWAYS, 0x0, 0xff);// se activeaza scrierea in stencil buffer

glStencilOp(GL_KEEP, GL_INCR, GL_KEEP); //se incrementeaza stencil buffer pt toate
// fragmentele care trec testul de adancime

***Rasterizeaza poligoanele de umbra**

Efectul:

Sunt rasterizate poligoanele de umbra de tip PUV.

Pentru fiecare fragment rezultat, f, se face testul:

daca f.z < z-buffer[f.y][f.x] // fragmentul f in fata fragmentului vizibil in pixelul (x,y)

atunci stencil-buffer[f.y][f.x]++;

// se numara poligoanele PUV aflate intre observator si obiectele scenei

VOLUME DE UMBRA (9)

Implementarea metodei folosind stencil-buffer(5)

Pasul 3

glCullFace(GL_FRONT);//se vor elimina feţele din faţă (nu sunt rasterizate poligoanele PUV)

glStencilOp(GL_KEEP, GL_DECR, GL_KEEP); //se decrementeaza stencil buffer pt toate
// fragmentele care trec testul de adancime

***Rasterizeaza poligoanele de umbra**

Efectul:

Sunt rasterizate poligoanele de umbra de tip PUN.

Pentru fiecare fragment rezultat, f, se face testul:

daca f.z < z-buffer[f.y, f.x] // f este in faţa fragmentului vizibil in pixelul (x,y)

atunci stencil-buffer[f.y, f.x]--; // se numara poligoanele PUN aflate intre observator
// si obiectele scenei

In stencil-buffer s-a obtinut o masca cu “gauri” in punctele care primesc lumina:

valorile din stencil-buffer sunt diferite de zero pentru pixelii care trebuie afisati in umbra.

VOLUME DE UMBRA (10)

Implementarea metodei folosind stencil-buffer(6)

Pasul 4

```
glEnable(GL_STENCIL_TEST);// Activeaza testul stencil
glStencilOp(GL_KEEP, GL_KEEP, KEEP); // Dezactiveaza scrierea in stencil-buffer
glStencilFunc(GL_EQUAL, 0x0, 0xff); // Specifica testul stencil
glDepthMask(GL_TRUE); // Activeaza scrierea in z-buffer
glColorMask(GL_TRUE, GL_TRUE, GL_TRUE, GL_TRUE); // Activeaza scrierea in frame
                                                 // buffer
```

***Rasterizeaza scena (folosindu-se algoritmul zbuffer) cu lumini**

Efectul:

Se rasterizeaza scena modificandu-se in frame-buffer numai culorile pixelilor (x,y) pentru care stencil-buffer [y][x] =0 (fragmentul nu este in umbra)

VOLUME DE UMBRA (11)

Implementarea metodei folosind stencil-buffer(7)

Cazul general:

Se initializeaza stencil-buffer cu o valoare reprezentand numarul de volume de umbra care contin pozitia observatorului.

Aprecieri asupra metodei volumelor de umbra

Pozitive:

- Este generală şi poate fi aplicată pentru orice scena, pentru oricate surse de lumina.
- Produce forme de umbre exacte.

Negative:

- Complexa din punct de vedere al timpului de calcul:

  se rasterizeaza scena de 2 ori si poligoanele de umbra o singura data dar in 2 pasi!

- Sursele de lumina sunt considerate punctiforme, de aceea nu pot fi obtinute penumbre.

Introducerea umbrelor în algoritmul z-buffer - shadow mapping -

Williams [WiLL78] a propus o metodă de generare a umbrelor bazată pe rasterizarea de 2 ori a scenei, cu execuţia algoritmului Z-Buffer:

- In prima etapă se construieşte buffer-ul Z al scenei văzută din poziţia sursei de lumina (în transformarea de vizualizare a scenei – funcția lookAt - se consideră poziţia sursei).

  Vom nota acest buffer cu ZS. El contine coordonatele z ale fragmentelor care primesc lumina de la sursa. Mai este numit si « **shadow map** », de unde si denumirea sub care este cunoscuta metoda in prezent.

- In etapa a doua se construieşte imaginea văzută din poziţia observatorului, utilizand buffer-ul ZS pentru determinarea punctelor aflate in umbra.

In cazul mai multor surse de lumină, se utilizează câte un buffer ZS pentru fiecare sursă.

Implementarea moderna a metodei este numita “shadow mapping”.

Introducerea umbrelor in algoritmul z-buffer

Etapa I.

Se calculeaza matricea de transformare a varfurilor, MVPS, cu observatorul in pozitia sursei de lumina

Se transmite MVPS la vertex shader (ca variabila uniforma).

Se cere folosirea unui buffer ZS ca buffer de adancime, in locul buffer-ului Z implicit

Se cere rasterizarea scenei:

1. In vertex shader se transforma varfurile primitivelor folosind MVPS
2. pentru fiecare primitiva a scenei

    pentru fiecare fragment f(x,y,z) rezultat din rasterizarea primitivei

        dacă z < ZS[y][x] atunci

            ZS[y][x] = z //fragmentul este vizibil în pixelul (x,y) din poziţia sursei

La sfarsitul acestei etape, in ZS sunt memorate coordonatele z ale fragmentelor care primesc lumina de la sursa.

Introducerea umbrelor în algoritmul z-buffer

Etapa II.

Se calculeaza matricea de transformare a varfurilor MVPO, cu observatorul in pozitia sa.

Se transmit la vertex shader (ca variabile uniforme) MVPO si MVPS.

Se cere rasterizarea scenei:

1. In vertex shader se calculeaza varful transformat cu MVPO (gl_Position) si MVPS (poz_sursa)

poz_sursa = pozitia varfului transformata cu MVPS este variabila de iesire din vertex shader

2. pentru fiecare primitiva a scenei

pentru fiecare fragment f(x,y,z) rezultat din rasterizarea primitivei

dacă z < Z-Buffer[y][x] atunci //fragment vizibil în pixelul (x,y), din poziţia observatorului

{Z-Buffer[y][x] = z; frame-Buffer[y][x] = Culoare fragment (întoarsa de fragment shader);}

In fragment shader

poz_sursa este variabilă de intrare pt fragm shader (pozitie fragment transformata cu MVPS)

Culoare_fragment = culoare calculata folosind modelul de iluminare locală;

dacă ZS[poz_sursa.y][poz_sursa.x] < poz_sursa.z atunci //fragmentul f(x,y,z) este în umbră

Culoare_fragment = factor_umbra* Culoare_fragment (0< factor_umbra<1)